

Operating Systems Study Notes - Chapter 5: Process Synchronization

1. Critical-Section Problem

A critical section is a piece of code that accesses shared resources (such as data structures or peripheral devices).

A solution to the critical-section problem must satisfy three requirements:

1. Mutual Exclusion: If process P_i is executing in its critical section, then no other processes can be executing in its critical section.
2. Progress: If no process is executing in its critical section and some processes wish to enter their critical section, then the system must allow one of them to do so.
3. Bounded Waiting: There must be a bound, or limit, on the number of times that other processes are allowed to enter their critical sections before the requesting process gets its turn.

2. Semaphores and Mutexes

A semaphore S is an integer variable that, apart from initialization, is accessed only through two standard atomic operations: wait and signal.

A mutex lock provides mutual exclusion using a boolean variable available indicating whether the lock is available.

Operating Systems Study Notes - Chapter 6: Deadlocks

1. System Model and Necessary Conditions for Deadlock

In a multiprogramming system, several processes may compete for a finite number of resources. A process requires

Deadlock can arise if four conditions hold simultaneously in a system:

1. Mutual Exclusion: At least one resource must be held in a nonshareable mode; that is, only one process at a time can use the resource.
2. Hold and Wait: A process must be simultaneously holding at least one resource and waiting to acquire additional resources held by other processes.
3. No Preemption: Resources cannot be preempted; that is, a resource can be released only voluntarily by the process holding it.
4. Circular Wait: A closed chain of processes exists such that each process holds at least one resource needed by another process in the chain.

2. Deadlock Prevention

Deadlock prevention provides a set of methods for ensuring that at least one of the four necessary conditions cannot hold.

Operating Systems Study Notes - Chapter 6 (Cont.): Deadlock Detection and Recovery

1. Deadlock Detection Algorithms

If a system does not employ either a deadlock-prevention or a deadlock-avoidance algorithm, then a deadlock situation can occur.

- An algorithm that examines the state of the system to determine whether a deadlock has occurred.
- An algorithm to recover from the deadlock.

For systems with single instances of each resource type, a wait-for graph variant of resource-allocation graphs is used.

For multiple instances of each resource type, the Banker-like detection algorithm uses Available, Allocation, and Request matrices.

2. Recovery from Deadlock

When a detection algorithm determines that a deadlock exists, several alternatives are available:

- Process Termination: Abort all deadlocked processes, or abort one process at a time until the deadlock cycle is eliminated.
- Resource Preemption: Successively preempt some resources from processes and give these resources to other processes.

Operating Systems Study Notes - Chapter 8: Memory Management & Paging

1. Paging Architecture

Paging is a memory-management scheme that permits the physical address space of a process to be noncontiguous.

The basic method for implementing paging involves breaking physical memory into fixed-sized blocks called frames.

Every logical address generated by the CPU is divided into two parts:

- A page number (p): used as an index into a page table. The page table contains the base address of each page.
- A page offset (d): combined with the base address to define the physical memory address that is sent to the memory.

2. Translation Lookaside Buffer (TLB)

A translation lookaside buffer (TLB) is a hardware cache that stores recent virtual-to-physical address translations.